Chapter 3

Canonical Evolutionary Algorithms

3.1 Introduction

A very simple evolutionary algorithm, EV, was introduced in chapter 1 to illustrate the basic ideas and issues involved in evolutionary computation. In this chapter we revisit EV, discuss its features in more detail, generalize it, and then use it to develop simple versions of the three historically important evolutionary algorithms: evolutionary programming, evolution strategies, and genetic algorithms. This leads naturally to a unified view of evolutionary computation that is discussed in detail in chapter 4.

3.2 EV(m,n)

Recall from chapter 1 that EV maintained a fixed-size population of individuals, each of which was a fixed-length vector (chromosome) of real-valued parameters (genes), and whose objective fitness was determined by calling a "landscape" evaluation function. After randomly generating and evaluating the fitnesses of the members of the initial population, EV simulates population evolution by repeatedly selecting a member of the current population at random (uniformly) as a parent to produce a clone offspring. EV then introduces some variation in the offspring via a simple mutation operator which, on average, picks one gene genome in the genome to modify by randomly adding/subtracting a small value to the current value of the selected gene. The new offspring is forced to compete immediately for survival against an existing member of the population selected at random (uniformly). If the objective fitness of the child is greater than the selected member, the child survives and the old member dies off. Otherwise, the child dies without ever residing in the population.

This form of population management in which offspring are produced one at a time and immediately compete for survival is called an incremental or "steady state" model. An alternate model is one in which a larger number (a batch) of offspring are produced from a fixed pool of parents. Only after the entire batch is produced is there competition for

survival into the next generation. Such models are often referred to as batch or generational algorithms.

It is easy to transform incremental EV into a batch system by explicitly maintaining two populations, one for m parents and one for n offspring. The two populations interact as follows:

```
For I=1 to n Do:
  Randomly select an individual from the parent population to
  produce a child as before, and add the child to the offspring
  population.

For I=1 to n Do:
  Force offspring I to compete for space in the parent population
  as before.
```

When described this way, it is easy see that the EV system introduced in chapter 1 is just a special case with $n = 1$. So from this point on it will be convenient to adopt the more general notation of $EV(m, n)$ in order to state explicitly the sizes of the parent and offspring populations. Conceptually, this generalization raises an interesting and difficult question: how large a batch of children should one produce before forcing them to compete for survival? Perhaps you have an idea or an opinion already! A good way to test your intuition right now is to experiment a bit with $EV(m, n)$ on the landscapes L1 and L2 that were introduced in chapter 1.

To set the stage for the rest of the chapter, let's observe the behavior of $EV(m, n)$ for one frequently chosen value of n, namely $n = m$ (i.e., identical sizes for both the parent and offspring populations). As we observed in chapter 1, the effects of such changes are easier to see at a macro-level. Figure 3.1 illustrates this by comparing the average fitness of the population that was evolved using both $EV(10, 1)$ and $EV(10, 10)$ on L2, the same 2-D multi-peaked fitness landscape that was used in chapter 1.

In this particular, case we observe $EV(10, 1)$ converging at a slightly faster rate to a region of high fitness. Is this what you expected? Is this true in general or is it just a property of these particular experiments? We will have a better understanding of this issue by the end of the chapter, but we will have to wait until chapter 4 to be more definitive.

We are now in a position to use $EV(m, n)$ as the basis for a more detailed look at some of the traditional evolutionary algorithms. To keep things simple I will continue to use the real-valued fitness landscapes, L1 and L2, and focus on the differences in the mechanics of evolution.

3.3 Evolutionary Programming

One observation about the design of $EV(m, n)$ is that it appears unnecessarily stochastic. By randomly selecting parents, it is quite possible that some members of the current population never produce any offspring in spite of the fact that they may have high fitness. Similarly, by randomly picking members from the current population to compete with children for

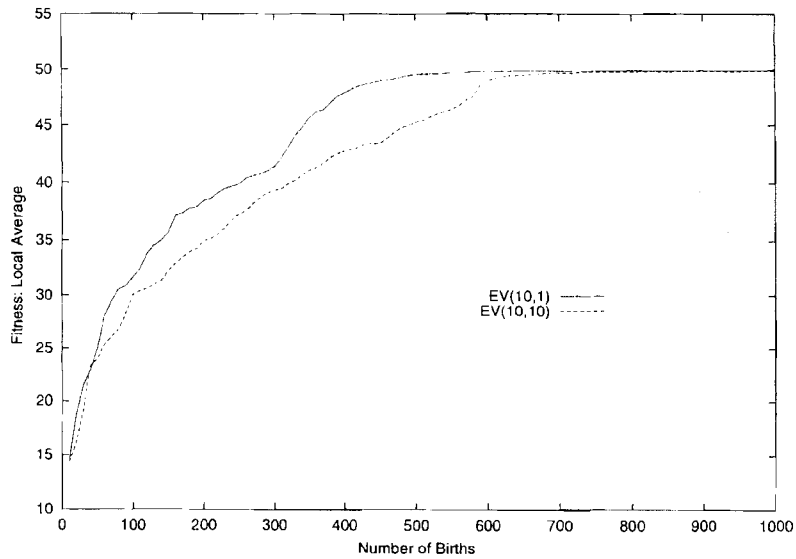

Figure 3.1: Average population fitness with different offspring population sizes on landscape L2.

survival into the next generation, it is quite possible that weak individuals may survive by the luck of the draw and not by merit.

It is easy to make EV more deterministic if we assume, as in the previous section, that both the parent and offspring populations are of equal size ($m = n$) and modify EV as follows:

```
For I=1 to m Do:
  Use parent I to produce 1 child and add it to the offspring
      population.
  End Do

From the combined pool of 2m parents and children, select only
     the m individuals with highest fitness to survive.
```

This is precisely the form that the early evolutionary programming (EP) algorithms took (Fogel et al., 1966). Unlike $EV(m, m)$ in which parents produce one offspring *on the average* but with high variance due to stochastic effects, parents in $EP(m)$ produce exactly one offspring. Unlike $EV(m, m)$ in which low fitness individuals have some small chance of survival, they have none in $EP(m)$.

What are the effects of these changes on evolutionary behavior? Figure 3.2 shows the effect on average fitness using L2, the 2-D landscape from chapter 1.

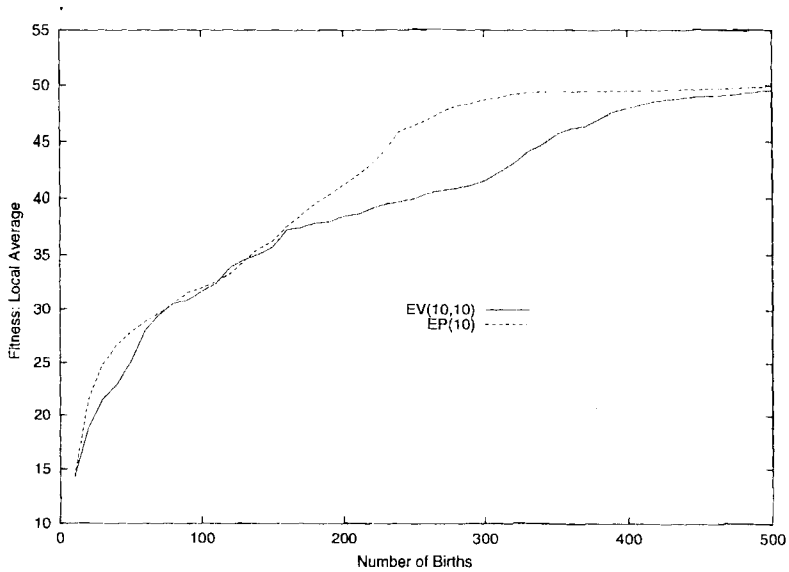

Figure 3.2: EV and EP average fitness on landscape L2.

In this case EP converges somewhat faster than EV to a region of high fitness. The reason for this should be fairly obvious: EP implements a much stronger "elitist" survival scheme in which only the top 50% survive, resulting in an increase in average fitness along with a decrease in diversity. Is this a useful property? It certainly appears to be on this particular 2-D landscape. But, as we will see later, this can also increase the likelihood of converging to a suboptimal peak. In general, achieving a proper balance between the components that increase variation (e.g., mutation) and those that decrease it (e.g., selection) is one of the most important factors in designing evolutionary algorithms.

The interested reader can find additional details about EP in Fogel (1995).

3.4 Evolution Strategies

Another observation about the EV and EP algorithms developed so far is that producing a very small number of offspring from a parent is not a very reliable sample of its potential for producing useful progeny. In nature organisms that produce only a few offspring are the exception rather than the rule. Computationally, it is quite easy to extend the EV and EP paradigms to encompass this idea by just allowing the size of the offspring population n to be greater than m, the size of the parent population. Having done so raises two interesting questions: how many offspring should each parent produce, and given that we are now using parents more efficiently, can we reduce the size of the parent population?

These ideas correspond directly to the early Evolution Strategy (ES) algorithms (Schwe-

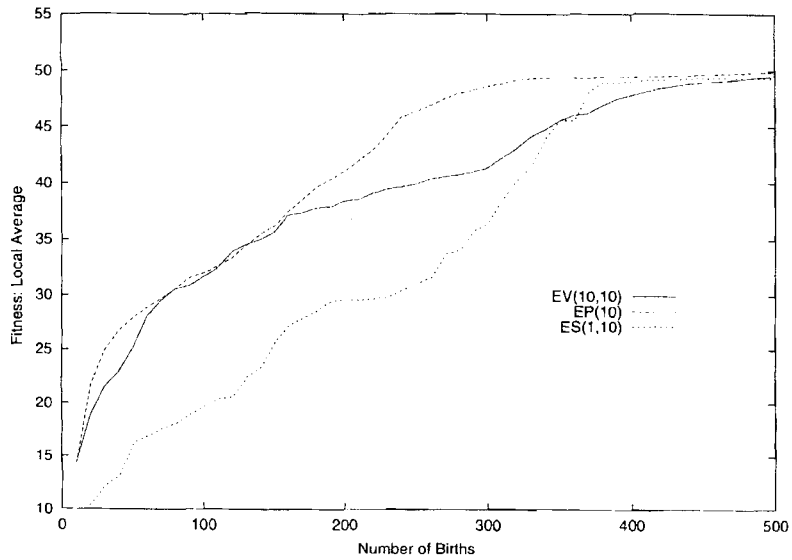

Figure 3.3: EV, EP, and ES behavior on landscape L2.

fel, 1975), which took the form of $(\mu + \lambda)$-ES models in which μ specified the size of the parent population and λ specified the size of the offspring population. Perhaps most striking about this early work was the focus on $(1+\lambda)$-ES models in which the entire next generation was produced from a single parent! For those who are familiar with Dawkins' work (e.g., Dawkins (1986)), this is also quite similar to the evolutionary model he uses in his examples.

To get a better feeling for the effects of these ideas on evolutionary behavior, we can run a $(1+10)$-ES model by simply running $EV(m,n)$, setting $m = 1$ and $n = 10$, and observing the differences between it and the previous EV and EP simulations. Figure 3.3 plots the observed effect on average population fitness.

The results are quite striking. One might expect that, since an $ES(1,10)$ is even more elitist than $EP(10)$, it would converge even faster. But this example illustrates nicely the critical role that the population plays, namely, providing a natural mechanism for parallel adaptive search. In this particular example, the initial randomly generated parent for the $ES(1,10)$ model happened to be not particularly close to any of the four peaks. From there, progress toward a peak can be made only via a long series of mutation increments. By contrast, the initial populations of both EV and ES had at least one randomly generated member reasonably close to a peak. After a few generations, selection abandoned the other parallel search threads and sampled almost exclusively in the areas near peaks. By simply increasing the ES parent population size, the behavior more closely approximates EV and EP as illustrated in figure 3.4.

Alternatively, one could view this as a fundamental problem with the mutation operator that we have been using, namely that it is controlled by a time-invariant Gaussian probability

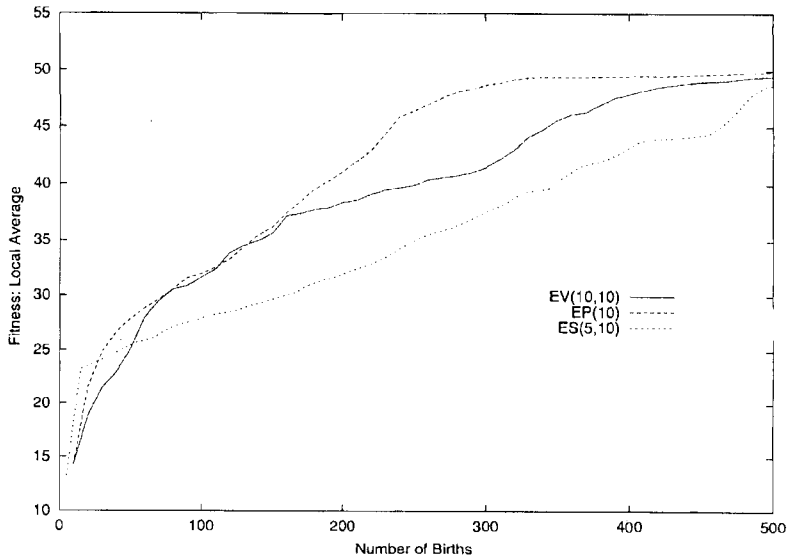

Figure 3.4: EV, EP, and ES average fitness on landscape L2.

distribution $G(0, \sigma)$ that *a priori* sets the average mutation step size. If this step size is small relative to the distances that need to be traveled, a long sequence of "successful" mutations (those that improve fitness) will be required. Conversely, if the average step size is too large, final convergence will be negatively affected by the lack of small step sizes. Figure 3.5 provides a simple illustration of these effects on average population fitness for mutation step sizes of 0.1, 0.5, and 5.0.

Such observations resulted in an early focus in the ES community on self-adapting the step size of the mutation operator (Rechenberg, 1965). Initial theoretical work on optimal rates of convergence (convergence velocity) resulted in an interesting 1:5 self-adaptation rule for mutation which states that if more than 20% of the mutations are successful using a given σ, then the step size is too conservative (exploitative) and σ needs to be increased. Similarly, if fewer than 20% of the mutations are successful, the step size is too explorative and needs to be reduced. More difficult to derive theoretically are "meta" parameters such as how often to adjust σ and by how much (Bäck and Schwefel, 1993; Schwefel, 1995).

To get an initial sense of how this affects the behavior of ES systems, consider the output of a modified version of $EV(m, n)$ that implements a simple version of the 1:5 rule in which σ is adjusted (if necessary) by 10% each generation. The effects of this adaptive step size feature are shown in figure 3.6.

In this example the mutation σ was set initially to 1.0. This value was classified by the 1:5 rule as too conservative and was increased for several generations, and then decreased continuously from there on. The overall effect is to improve both initial and final convergence rates.

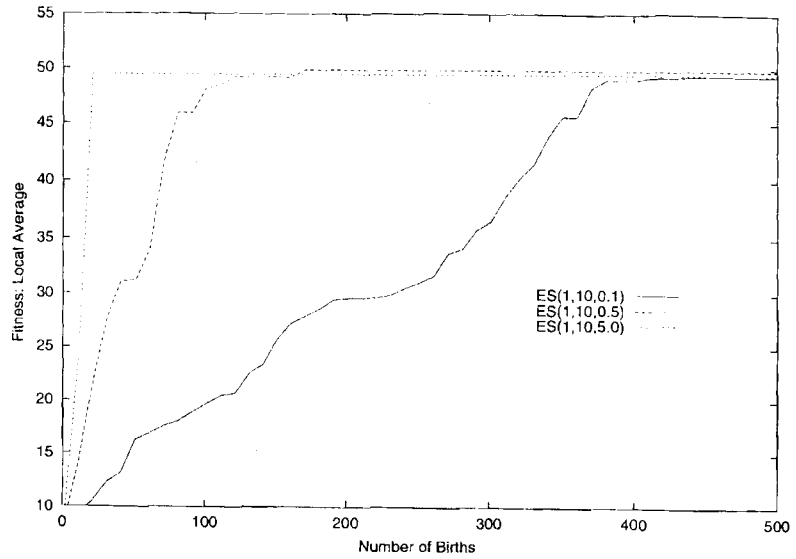

Figure 3.5: The effects of mutation step size on landscape L2.

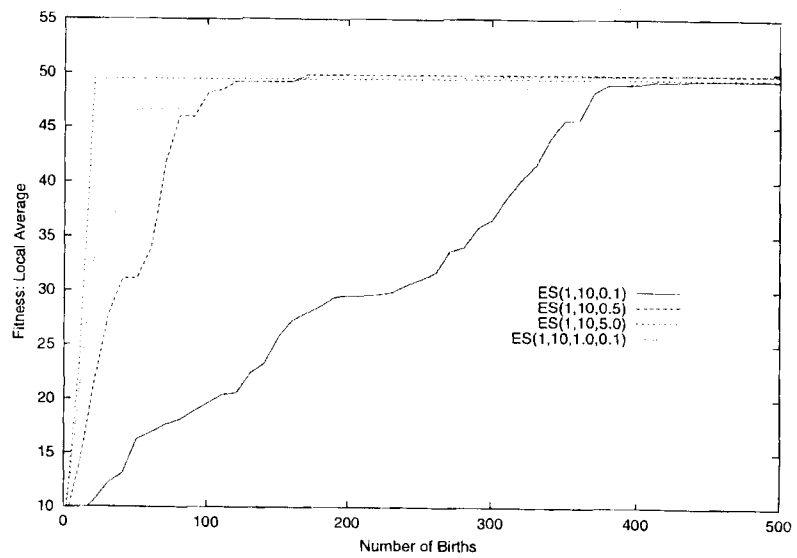

Figure 3.6: The effects of an adaptive mutation step size on landscape L2.

As one might expect, things get more complicated for higher-dimensional problems in which it may be beneficial to maintain a separate step size for each dimension. If there are strong interactions among the parameters of the search space, simultaneous correlated mutations may prove useful. These ideas will be explored in more detail in later chapters.

Self-adaptation can also be viewed as the simultaneous evolution (the coevolution) of both the objects of interest (in this case parameter vectors) and the operators required to evolve them. How one designs and implements coevolutionary systems is a fascinating but difficult topic that will be deferred to a later chapter.

The interested reader can find many additional details about ESs in Rechenberg (1994) and Schwefel (1995).

3.5 Genetic Algorithms

An important observation regarding the evolutionary systems we have studied so far is that individuals die off only when replaced by younger individuals with higher fitness. Evolutionary systems of this type exhibit the convenient property that local population statistics, such as maximum, minimum, and average fitness, are monotonically non-decreasing functions of time, resulting in nice mathematical convergence properties. However, such systems are neither biologically plausible nor computationally desirable in cases in which the fitness landscapes are more complex than the simple ones that have been used so far. Intuitively, allowing individuals to survive and reproduce indefinitely can result in a significant loss of diversity in the population and can increase the likelihood of becoming trapped on a false peak.

There are a number of ways one can address this problem. One approach is to occasionally allow new individuals to replace existing individuals with higher fitness. A more direct method is to use a generational model in which parents survive for exactly one generation and are completely replaced by their offspring. This is the form that standard genetic algorithms (GAs) take (Holland, 1975; De Jong, 1975; Goldberg, 1989), and also the form that the (μ, λ)-ES models take (Bäck and Schwefel, 1993; Schwefel, 1995).

GAs also implement the biological notion of fitness (the ability to survive *and* reproduce) in a somewhat different fashion than we have seen so far. The EV, EP, and ES systems all use objective fitness to determine which offspring survive to adulthood (i.e., which ones make it into the parent population). However, once in the parent pool, there is no additional bias with respect to which individuals get to reproduce – all parents have an equal chance.

Alternatively, one could allow *all* offspring to survive to adulthood and then use objective fitness to select which parents are allowed to reproduce. This is precisely the form that standard GAs take, namely:

```
GA:

Randomly generate a population of m parents.

Repeat:
    Compute and save the fitness u(i) for each individual i
        in the current parent population.
```

```
Define selection probabilities p(i) for each parent i
    so that p(i) is proportional to u(i).

Generate m offspring by probabilistically selecting parents
    to produce offspring.

Select only the offspring to survive.

End Repeat
```

Notice that the pseudo-code actually specifies a particular form of selection, called fitness-proportional selection. It is easy to show that using this selection mechanism, individuals in the current population with average fitness will produce on average one offspring, while above-average individuals produce more than one and below-average individuals less than one (Holland, 1975).

Figure 3.7 illustrates the effect that this has on the average fitness of an evolving population. Notice first that fitness-proportional selection appears to be stronger initially than that of ES, but then weakens considerably. The second, and more striking, outcome to note is that, since all parents die off after one generation, it is possible for the average fitness of the next generation (the offspring) to drop. Is this desirable? For simple landscapes, probably not. But for more complex, multipeaked landscapes this can decrease the likelihood of converging to a local optimum.

3.5.1 Multi-parent Reproduction

Another important difference between GAs and the other models is the way in which offspring are produced. From the beginning, Holland stressed the importance of sexual reproduction as a source of variation in the offspring produced (Holland, 1967). The basic idea is that offspring inherit gene values from more than one parent. This mixing (recombining) of parental gene values, along with an occasional mutation, provides the potential for a much more aggressive exploration of the space. Holland's initial ideas for this took the form of a "1 point crossover" operator in which a randomly chosen initial segment of genes was inherited from one parent and the remaining segment inherited from a second parent. These ideas were later generalized to "2 point crossover" and "n point crossover" operators (De Jong, 1975).

Since all of these crossover operators are equivalent for individuals with exactly two genes, we run a GA with crossover (GA-X) on landscape L2 and plot the effects on average population fitness in figure 3.8.

In this case, crossover provides an additional source of variation involving larger initial steps, improving the initial rate of convergence. Since crossover does not introduce new gene values, its influence diminishes as the population becomes more homogeneous, and the behavior of GA-X becomes nearly identical to a GA with no crossover.

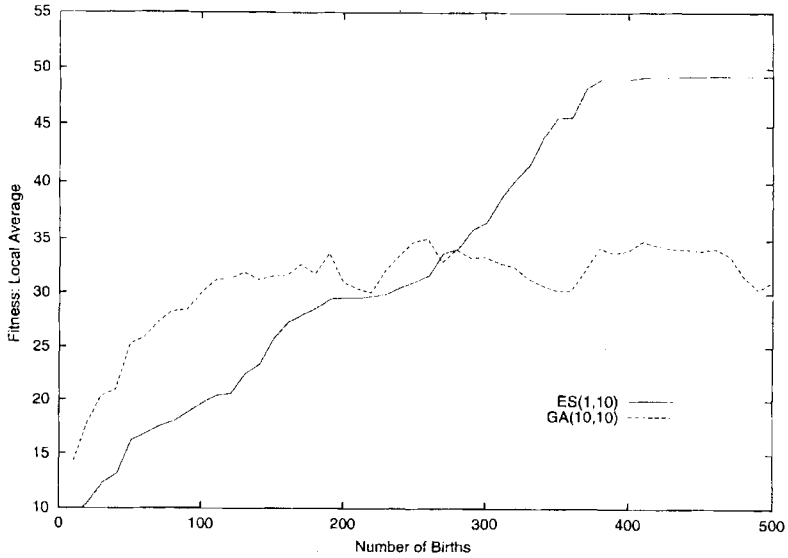

Figure 3.7: Average population fitness of a GA on landscape L2.

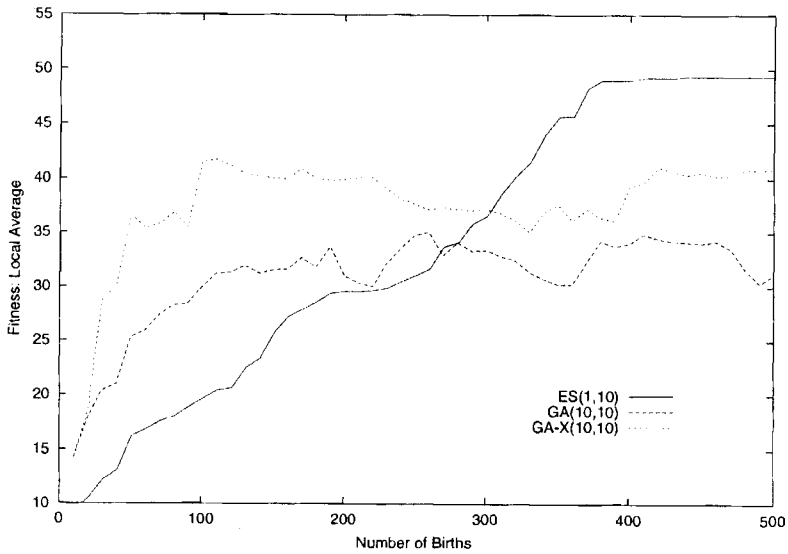

Figure 3.8: Average population fitness of a GA with crossover on landscape L2.

3.5.2 U

Holland als
to be used
one can foc
requiring n
initially sug
gene, and t
Crossover a
much finer
to a "bit fli

To make
mization pr
this is to re
So, for a 1-
bounds of [0
each point c
apart. We c
internal l bit
ith gridpoin
$v = lower_b$

To get a
produced on
resolution of

```
Simula
Random
Using
  with
    1
  and
    1
EA typ
Crosso
Using
Parent

Popula
 Globa
 Local
 Indiv
    1
    2
    3
    4
    5
```

3.5.2 Universal Genetic Codes

Holland also emphasized the importance of a universal string-like "genetic" representation to be used internally by a GA to represent the genome of an individual. By doing so, one can focus on a single problem-independent set of reproduction operators, rather than requiring new ones for each new problem-specific representation. For simplicity, Holland initially suggested a binary string representation in which each bit is viewed internally as a gene, and the mapping to the external phenotype is left unspecified and problem specific. Crossover and mutation operate as before at the gene level, except that they do so at a much finer level of granularity. In the case of a binary representation, mutation simplifies to a "bit flipping" operator.

To make this more concrete, consider how one might map real-valued parameter optimization problems into an internal binary representation. A standard way of accomplishing this is to require parameter resolution information in addition to upper and lower bounds. So, for a 1-D landscape one might specify a resolution of 10^{-4} in addition to parameter bounds of $[0.0, 5.0]$. This permits a calculation of the number of bits required to represent each point of a grid uniformly spaced on the interval $[0.0, 5.0]$ with the grids points 10^{-4} apart. We can then easily compute the corresponding real value v of a parameter from its internal l bit representation by interpreting the l bits as a binary number i representing the ith gridpoint from the left endpoint (lower bound) of the parameter interval, and letting $v = lower_bnd + i * resolution$.

To get a sense of the effect of an internal binary representation, consider the output produced on the 1-D landscape defined by $f(x) = x^2$ over the interval $[0.0, 5.0]$ with a resolution of 10^{-4}.

```
Simulation time limit (# births): 500
Random number seed (positive integer): 12345
Using a parabolic landscape defined on 1 parameter(s)
  with parameter initialization bounds of:
     1: 0.0  5.0
  and with hard parameter bounds of:
     1: 0.0  5.0
EA type: ga
Crossover: one_point
Using a genome with 16 binary genes.
Parent population size: 10; offspring population size: 10

Population data after 10 births (generation 1):
 Global fitness:  max =  21.02047, ave =   8.96513, min =   0.37711
 Local fitness:   max =  21.02047, ave =   8.96513, min =   0.37711
 Indiv  birthdate   fitness      gene values
    1       1        5.50103  0111100000010110
    2       2        4.25183  0110100110010011
    3       3        8.62828  1001011001100101
    4       4       14.35989  1100001000000101
    5       5        0.41238  0010000011100001
```

6	6	12.75269	1011011011010111
7	7	20.78745	1110100101110000
8	8	0.37711	0001111101110001
9	9	21.02047	1110101010111110
10	10	1.56021	0011111111110100

In this case, we see that to achieve the desired range and resolution, 16-bit strings were required. An initial population of randomly generated bit strings was generated and fitness was calculated by interpreting each bit string as a real number in the interval $[0.0, 5.0]$. One-point crossover and bit-flip mutation are used at the string level to produce the next generation. Notice that small changes at the bit string level can result in rather large changes in the corresponding real number (e.g., flipping the leftmost bit). Continuing, we observe definite patterns emerging in the binary gene pool:

```
Population data after 80 births (generation 8):
 Global fitness:  max =  23.88882, ave =  16.86300, min =   0.03412
 Local fitness:   max =  22.52034, ave =  20.34158, min =  12.28837
 Indiv  birthdate   fitness      gene values
    1      71       21.45503  1110110100101000
    2      72       21.27589  1110110000101010
    3      73       12.28837  1011001101111011
    4      74       21.50737  1110110101110010
    5      75       22.38297  1111001000111011
    6      76       21.45503  1110110100101000
    7      77       22.52034  1111001011111001
    8      78       21.62713  1110111000011011
    9      79       21.34703  1110110010001111
   10      80       17.55663  1101011010001000
```

```
Population data after 240 births (generation 24):
 Global fitness:  max =  24.84080, ave =  17.78426, min =   0.03412
 Local fitness:   max =  24.73368, ave =  18.59314, min =  11.17597
 Indiv  birthdate   fitness      gene values
    1     231       23.16574  1111011001101110
    2     232       21.19713  1110101110111010
    3     233       11.17597  1010101100101010
    4     234       24.06290  1111101100101000
    5     235       21.72872  1110111010101010
    6     236       24.73368  1111111010100010
    7     237       12.31353  1011001110101010
    8     238       21.79706  1110111100001010
    9     239       11.70443  1010111100101010
   10     240       14.05220  1011111111101110
```

```
Population data after 470 births (generation 47):
 Global fitness:  max =  24.94434, ave =  17.66376, min =   0.05648
 Local fitness:   max =  24.65861, ave =  21.53086, min =  10.52465
 Indiv  birthdate   fitness      gene values
    1     461       24.65861  1111111000111111
    2     462       20.20855  1110011000101010
    3     463       23.11069  1111011000100011
    4     464       10.52465  1010011000011010
    5     465       24.63286  1111111000011101
    6     466       23.13123  1111011000111111
    7     467       23.87465  1111101000101100
    8     468       23.12756  1111011000111010
    9     469       18.17187  1101101001000010
   10     470       23.86794  1111101000100011

Global best:  birthdate   fitness      gene values
               346       24.944336  1111111110110111
```

For this particular 1-D landscape, the real numbers with high fitness are those at the right-most end of the interval $[0.0, 5.0]$. In binary form, these are real numbers whose binary string representations have lots of leading 1s, and the highest fitness is associated with the string of all 1s. Fairly early in the simulation we see 1s beginning to dominate the left-most gene positions, and we note that a near-optimal string is already encountered at time 346. Recall, however, that this GA is a generational model, so even the best individuals only live for one generation. After about 400 births this GA settles into a dynamic equilibrium in which the diversity provided by crossover and mutation is balanced by the selection pressure toward homogeneity.

An obvious question is whether there is any significant difference between using a binary or a real encoding. A detailed answer will have to be deferred to later chapters. However, it is easy to illustrate that there are significant differences. This can be seen in a number of ways. First, consider its effect on the average fitness of the population. Figure 3.9 compares the average fitness of a binary-coded GA (GA-B) with its real-coded equivalent(GA-R) on landscape L2. Notice how the larger steps that are possible in a binary GA provide some initial advantage, but then have a negative effect.

But, of course, average population fitness is only one aspect of evolutionary behavior. From a computation search perspective, best-so-far curves provide a better picture of search performance. Figure 3.10 illustrates this aspect for GA-B and GA-R on landscape L2. Notice how, in this case, we see just the opposite. In spite of lower average fitness, GA-B is able to find individuals with higher fitness faster than GA-R.

The interested reader can find many additional details about GAs in Goldberg (1989), Holland (1992), and Mitchell (1998).

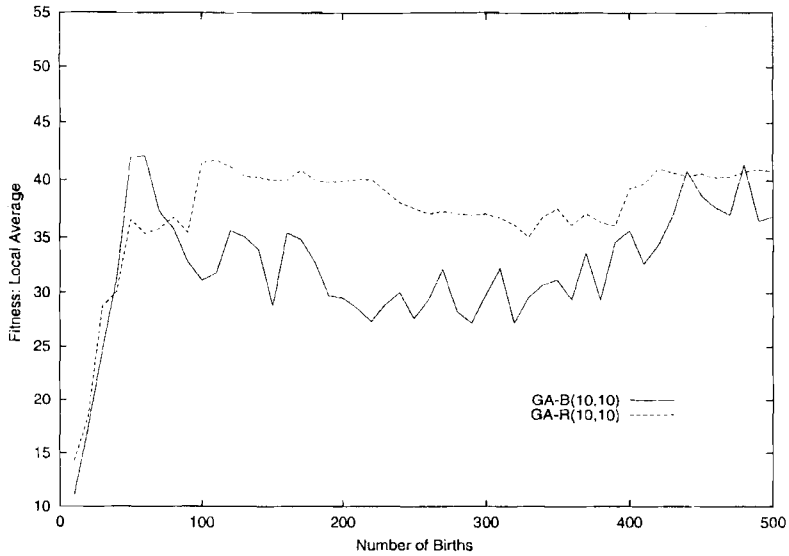

Figure 3.9: Average population fitness of real and binary coded GAs on landscape L2.

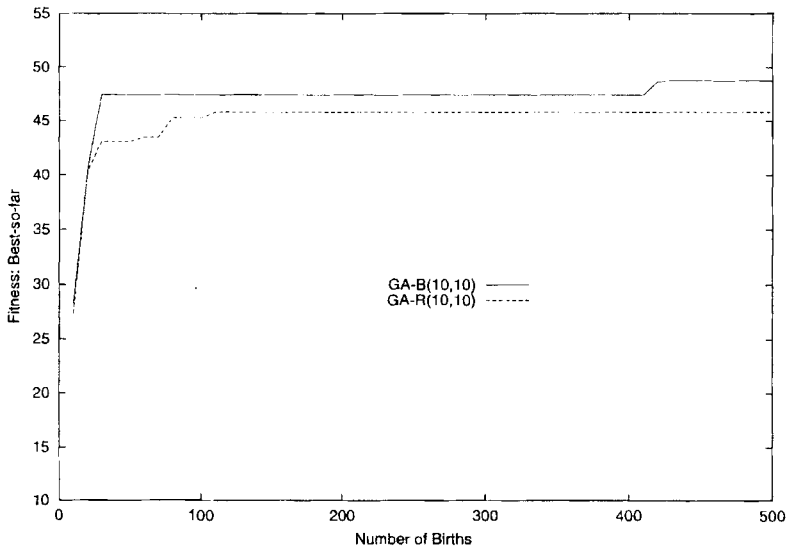

Figure 3.10: Best-so-far curves of real and binary coded GAs on landscape L2.

3.6 Summary

The canonical evolutionary algorithms EP, ES, and GA are important for several reasons. First, they have had a strong influence on the field of evolutionary computation as we see it today. A majority of the evolutionary computation applications that have been developed over the past 30 years have been strongly influenced by these canonical EAs. A second role that they play is to emphasize that there are a variety of ways in which interesting and useful evolutionary algorithms can be designed and used for problem solving. Finally, these canonical EAs serve as the basis for a more general framework that can help us build new and more effective evolutionary problem solvers. These ideas are explored more fully in the next several chapters.

L2.